

Personalized GPT Portfolio – Detailed Design, Rationale, and Deployment Plan

0) Executive Summary

A **Personalized GPT Portfolio** ("SagarGPT") lets recruiters and collaborators chat with your profile: ask about skills, projects, metrics, and get **grounded, cited answers** pulled from your resume, cover letters, LinkedIn, GitHub, portfolio pages, and work docs.

This document explains **what we're building, why each technology is chosen, how to deploy the final website**, and **how to make it configurable so anyone can create their own Personal GPT** by uploading their materials.

1) Problem, Goals, and Non-Goals

Problem. Static resumes are hard to query; recruiters need quick, truthful answers and evidence.

Goals. - Truthful Q&A grounded in your verified documents (no hallucinations). - Fast "yes/no + evidence" skill checks. - Easy to embed into a portfolio website as a chat dock. - Manageable and configurable for **multiple users** (multi-tenant option).

Non-Goals. - Not a general web agent that fills forms by default. (Optional later.) - Not replacing ATS; it complements it.

2) System Overview

User (Recruiter) ↔ Portfolio Site (Next.js) ↔ RAG API (FastAPI) ↔ Vector DB (pgvector/Qdrant)



Object/Doc Store (S3/Supabase)
Relational DB (Postgres/Supabase)
Reranker Model (cross-encoder)
LLM API / Local LLM (Ollama)

3) Design Choices — What & Why

3.1 Retrieval-Augmented Generation (RAG)

- **What:** Retrieve the most relevant doc chunks for a query, then generate an answer strictly from those chunks, with citations.
- **Why:** Keeps answers **truthful** and **explainable**; avoids risky end-to-end finetuning over private data.

3.2 Hybrid Search (BM25 + Embeddings) + Reranker

- **What:** Use term search (BM25) + semantic vectors to cast a wide net; then rerank top candidates with a **cross-encoder**.
- **Why:** Hybrid improves recall; reranker improves precision → fewer off-topic answers.

3.3 Vector Database (Supabase pgvector or Qdrant)

- **What:** Store chunk embeddings with metadata (source, section, tags, dates).
- **Why:** Fast k-NN search, scalable, production-ready. **Supabase** (pgvector) is simplest when you already need Postgres; **Qdrant** is great for pure vector workloads and high performance.

3.4 Embedding Models (bge-large / e5-large or hosted embeddings)

- **What:** Strong open embedding models or hosted embeddings for higher quality.
- **Why:** Better recall/precision → better answers. Open models allow local/offline privacy; hosted provide state-of-the-art with minimal ops.

3.5 Answer LLM (hosted vs local)

- **Hosted (Recommended for MVP):** Best quality, low ops; rate-limit and cache.
- **Local (Ollama + GPU):** Maximum privacy/control; requires good hardware for quality.
- **Why:** Start hosted for speed; keep a drop-in option to swap to local later.

3.6 Backend: FastAPI (Python)

- **What:** RAG orchestration service exposing `/ask`, `/skills`, `/sources`, `/ingest` endpoints.
- **Why:** Python ecosystem for NLP (pdfplumber, docx, BeautifulSoup, LlamaIndex/LangChain), quick dev, clean type hints, good perf with Uvicorn/Gunicorn.

3.7 Frontend: Next.js + Tailwind + shadcn/ui

- **What:** Portfolio site + floating chat widget, modes (Short/Detailed/STAR), citations UI.
- **Why:** Modern SSR/ISR, easy theming, great developer tooling, deploys well on **Vercel**.

3.8 Storage: Supabase (Postgres + S3-compatible storage) or AWS S3 + RDS

- **What:** Store raw docs, chunked text, user profiles, skill matrices.
- **Why:** Fewer moving parts if you use Supabase; or compose AWS services if you prefer cloud primitives.

3.9 Observability: Langfuse (tracing), Structured App Logs

- **What:** Capture prompts, retrieval context, latencies, user feedback.
- **Why:** Essential for improving retrieval quality and debugging.

3.10 Security & Privacy

- **Transport:** TLS everywhere.
 - **At Rest:** Encrypted DB volumes and object storage.
 - **PII:** Redact emails/phones if needed before indexing.
 - **Access:** API keys in secret manager; per-tenant RBAC and row-level security (RLS) where possible.
-

4) Data Model & Indexing

4.1 Chunk Schema (Vector Store)

- `id`: uuid
- `tenant_id`: org/user workspace id
- `doc_id`: source document id
- `source_type`: resume | cover_letter | linkedin | github | portfolio | paper | misc
- `title`, `section`, `url/path`
- `text`: content
- `embedding`: vector
- `tags`: JSONB {skills:[], metrics:[], roles:[], dates:{from,to}}
- `visibility`: public | private

4.2 Relational Tables

- `tenants` (`id`, `name`, `plan`, `theme`, `created_at`)
- `users` (`id`, `tenant_id`, `role`, `email`, `hashed_password`)
- `documents` (`id`, `tenant_id`, `source_type`, `title`, `url`, `storage_key`, `status`)
- `chunks` (`id`, `tenant_id`, `doc_id`, ...  vector index)
- `skills` (`id`, `tenant_id`, `name`, `synonyms[]`)
- `skill_evidence` (`id`, `tenant_id`, `skill_id`, `chunk_id`, `confidence`, `last_seen`)
- `queries` (`id`, `tenant_id`, `question`, `top_chunks[]`, `answer`, `latency_ms`, `created_at`)

4.3 Ingestion Pipeline

1) Upload/Fetch → 2) Parse (PDF/DOCX/HTML/Markdown) → 3) Clean → 4) Chunk (e.g., 800/200) → 5) Embed → 6) Persist vectors+meta → 7) Skill extraction (NER/keyword → evidence links).

Why these steps? Clean chunking and metadata tagging directly boost retrieval quality, grounding, and the UX for skills lookups.

5) Skills Matrix (Fast Yes/No with Evidence)

Precompute mapping `skill → [evidence chunks]` with confidences and links. Used by the chat to quickly answer: “Do you have CUDA?”

Why: Recruiters want immediate, verifiable answers with links; avoids waiting for full RAG every time.

6) Prompting & Guardrails

- **Instruction:** “Answer **only** from retrieved evidence. If missing, say you don’t have it or you’re open to learning. Include citations.”
 - **Style memory:** Tone, brevity, preferred bullet format.
 - **Filters:** Block unsafe requests; remove PII from outputs where required.
 - **Why:** Ensures honesty, clarity, and consistent voice.
-

7) Final Website & UX

- **Portfolio (Next.js)** with:
 - Floating **Chat Dock** (“Chat with Sagar’s profile”).
 - **Skill Chips** (C/C++, Linux, ROS2, CUDA, PyTorch, Embedded, SerDes, PoE, IoT, ML, CV).
 - Answer layout: **Yes/No + Level → Proof bullets → Source links**.
 - **Modes:** Short / Detailed / STAR story.
 - **Downloads:** Embedded / Robotics / Research resume variants.
 - Optional **Share/QR** for recruiters.

Why these UX details? They minimize friction, increase trust (citations), and show breadth quickly.

8) Deployment Options & Rationale

Option A: Hosted (Recommended MVP)

- **Frontend:** Vercel (static + server components) → CDN, preview deploys, easy custom domain.
- **Backend:** Railway/Render/Fly.io (Dockerized FastAPI) → simple scaling, logs.
- **DB & Vectors:** Supabase (Postgres + pgvector) → one platform for auth, SQL, and vectors.
- **Object Storage:** Supabase Storage (or S3) → stores PDFs/HTML/MD.
- **LLM/Embeddings:** Hosted API → best quality, low ops.
- **Observability:** Langfuse cloud.

Why: Minimal DevOps, rapid iteration, production-grade.

Option B: Private / Local

- **Docker Compose:** FastAPI, Qdrant, Postgres, Ollama, MinIO.

- **GPU VM:** Runpod/Lambda for strong local models.
- **Why:** Data never leaves your infra; useful for sensitive docs.

Option C: Cloud-Native (AWS reference)

- **Frontend:** Vercel or Amplify
- **Backend:** ECS Fargate or Lambda + API Gateway
- **DB:** RDS Postgres (pgvector) or Aurora + Qdrant on ECS
- **Storage:** S3 (+ CloudFront)
- **Secrets:** AWS Secrets Manager
- **Why:** Enterprise control, VPC, IAM, scaling.

9) Multi-Tenant: “Anyone Can Make Their Own Personal GPT”

9.1 Tenancy Model

- **Workspace (tenant) per user:** Separate indices and storage keys.
- **Row Level Security (RLS)** in Postgres; **namespace per tenant** in Qdrant.
- **API keys** scoped to tenant; user roles: `owner`, `editor`, `viewer`.

9.2 Onboarding Flow

1) Sign up / create workspace. 2) Upload documents or connect LinkedIn/GitHub/Google Drive. 3) Choose theme (colors/logo), tone presets, skill taxonomy. 4) Click **“Build my GPT”** → background ingestion job indexes data. 5) Get **embed code** (JS snippet) to add a chat dock to any site.

9.3 Configuration — Example JSON/YAML

`tenant.config.json`

```
{
  "name": "SagarGPT",
  "theme": {"primary": "#2563eb", "mode": "dark"},
  "tone": "concise-technical",
  "skills": ["C", "C++", "Linux", "ROS2", "CUDA", "PyTorch"],
  "connectors": {
    "linkedin": {"url": "https://linkedin.com/in/..."},
    "github": {"repos": ["sagar/EdgeSim.AI", "sagar/OpenBite"]},
    "portfolio": {"sitemap": "https://sagar.dev/sitemap.xml"}
  },
  "retrieval": {"hybrid": true, "reranker": true, "k": 8},
  "privacy": {"pii_redact": true, "public_answers": true}
}
```

Why JSON config? Easy to version, import/export, and drive UI defaults.

9.4 Admin Console

- Uploads, connector status, re-index button.
- Skill taxonomy editor (add synonyms).
- Prompt/tone presets.
- Logs: unanswered queries → suggest adding sources.
- Rate limits / plan quotas.

9.5 Embeddable Widget

- JS snippet adds a floating chat button to any site; tenant-scoped API key.
- Themeable; supports “question hints.”

9.6 Billing & Plans (Optional)

- Free: limited docs, rate limits, watermark.
- Pro: more docs, custom domain, analytics, resume variants.
- Team: shared workspace, role permissions.

10) APIs & Endpoints (Sketch)

- `POST /ingest` — upload doc or connector task → job id.
- `GET /ingest/:jobId` — job status.
- `POST /ask` — {question, mode} → {answer, citations, latency}.
- `GET /skills?name=CUDA` — quick yes/no + evidence.
- `GET /sources/:chunkId` — raw text + URL.
- `POST /tenant` — create workspace; returns embed key.

Why these endpoints? Minimal surface area, clear separation of concerns.

11) CI/CD & Operations

- **GitHub Actions:** lint, tests, container build, deploy to Vercel/Railway.
 - **Migrations:** Alembic (Python) or Prisma (TS) for schema.
 - **Backups:** Nightly DB & object storage snapshots.
 - **Caching:** Redis for retrieval/LLM responses when safe.
 - **Rate limiting:** Token bucket per tenant.
-

12) Security, Privacy, and Compliance

- **Least-privilege** DB users; RLS enforced.
- **Secrets** in managed vault (Vercel/Platform Secrets or AWS Secrets Manager).
- **Audit log** of admin actions.

- **Data deletion** self-serve; scrub on tenant delete.
 - **PII policy** and ToS surfaced at signup.
-

13) Cost & Scaling Notes

- **Startup (MVP):** Vercel Hobby + Railway Starter + Supabase Pro → low monthly cost.
 - **Hot paths:** Embedding & reranking → batch, cache, reuse.
 - **Spiky traffic:** Autoscale API service; add CDN edge cache for static assets.
-

14) Implementation Roadmap

1) **Week 1:** Ingest + vector store + `/ask` with citations. 2) **Week 2:** Skills matrix + Next.js widget + hosted deployment. 3) **Week 3:** Tenant onboarding + embed script + admin console basics. 4) **Week 4:** Observability, rate limits, polish, docs.

15) Deployment Checklist (Hosted MVP)

- [] Provision Supabase project (Postgres + Storage + pgvector).
 - [] Deploy FastAPI on Railway (set `DATABASE_URL`, `SUPABASE_KEY`, `LLM_KEYS`).
 - [] Run DB migrations (tenants, documents, chunks, skills, queries).
 - [] Configure Langfuse keys; ship trace hooks.
 - [] Deploy Next.js portfolio on Vercel; set `RAG_API_URL`, `TENANT_KEY`.
 - [] Test ingestion with sample docs; verify citations and skills answers.
 - [] Generate and share **embed snippet**.
-

16) Appendix A – Example Prompts

System: “You are SagarGPT. Answer **only** from retrieved evidence. Include citations. If not found, say so briefly and suggest the nearest related evidence.”

User: “Do you have ROS2 experience?” **Assistant:** “Yes — ROS2 bring-up in EdgeSim.AI (link), ROS2 nodes for ... (resume section link).”

17) Appendix B – Example Embed Snippet

```
<script>
  (function(){
    const s=document.createElement('script');
```

```
s.src="https://cdn.personal-gpt.dev/widget.js";
s.onload=()=>window.PersonalGPT.init({
  tenant:"TENANT_PUBLIC_KEY",
  api:"https://api.personal-gpt.dev",
  theme:{primary:"#2563eb", mode:"dark"},
  hints:["Top skills","Show robotics projects","Any CUDA?"],
});
document.head.appendChild(s);
})();
</script>
```

18) Conclusion

This stack balances **quality, trust, and speed to deploy**. RAG ensures evidence-backed answers; the skills matrix gives instant yes/no with citations; Next.js + FastAPI + Supabase makes it simple to host. The **multi-tenant** design and admin console make it **manageable and configurable**, so **anyone can create their own Personalized GPT portfolio** by uploading documents and connecting profiles.